DAA Practical 1

Fibonacci Series and Step Count

Top 60 Viva Questions & Answers

Core formulas

Fibonacci: F(n) = F(n–1) + F(n–2), with F(0)=0 and F(1)=1.
Loop iterations: n–2 for n≥2; zero iterations for n=0 or 1.
Printed stepCount in this code: n–2 for n≥2; it increments once per generated term/addition.
Time complexity: O(n). **Auxiliary space:** O(1).

Important step-count clarification

The program's variable `stepCount` is incremented exactly once inside each loop iteration. Therefore, for n=6 it prints 4. If a question separately asks you to count four selected body statements per iteration, that chosen statement-count model gives 4(n–2)=16 for n=6. These are different counting conventions. Do not confuse either count with Big-O: both lead to linear time, O(n).

Use this sheet for revision after understanding the code and dry run.

A. Fibonacci fundamentals and code logic

Q1. What is the Fibonacci series?

It is a sequence in which each term after the first two is the sum of the previous two terms: 0, 1, 1, 2, 3, 5, 8, 13, ...

Q2. What is the mathematical recurrence relation?

F(n) = F(n–1) + F(n–2), with starting values F(0) = 0 and F(1) = 1.

Q3. Why is addition used in this program?

Addition is the defining rule of the Fibonacci sequence: every new term is obtained by adding the two immediately preceding terms.

Q4. Why are the initial values a = 0 and b = 1?

They are the first two Fibonacci numbers. The program uses them to generate all later terms.

Q5. What do a, b and c represent?

a stores the previous Fibonacci term, b stores the current term, and c temporarily stores the next term.

Q6. Why does the program need a third variable c?

It preserves the sum a + b before the previous values are shifted. Without a temporary variable, an update could overwrite a value needed for the calculation.

Q7. Why are a = b and b = c performed after calculating c?

They move the pair forward by one position: the old current term becomes the previous term, and the newly calculated term becomes the current term.

Q8. What happens if the update statements are reversed or omitted?

The program may reuse stale values or lose a required previous term, so it will not generate the correct sequence.

Q9. Why are the first two terms printed using separate if statements?

They are already known as 0 and 1, so they do not need to be calculated by the loop. The conditions also handle requests for fewer than two terms.

Q10. Why does the loop start at i = 2?

The first two terms have already been printed. The loop calculates terms beginning with the third term, whose zero-based index is 2.

Q11. Why is the loop condition i < n?

The requested number of terms is n. With i starting at 2, the loop covers indices 2 through n–1, producing n–2 new terms.

Q12. How many times does the loop execute?

For integer $n \geq 2$, it executes n–2 times. For n = 0 or 1, it executes zero times.

Q13. What is the output when n = 1?

The program prints only 0 and the step counter remains 0.

Q14. What is the output when n = 2?

The program prints 0 1 and the step counter remains 0.

B. Dry run, step count and complexity

Q15. What is the output when n = 6?

It prints 0 1 1 2 3 5. The loop performs four additions, so the program's stepCount is 4.

Q16. Show the dry run for n = 6.

Initial a=0, b=1, stepCount=0. Iterations: 0+1=1; 1+1=2; 1+2=3; 2+3=5. The generated terms are 1, 2, 3, 5 and the counter ends at 4.

Q17. What is the step count recorded by this exact program?

The variable stepCount is incremented once per loop iteration. Therefore it records the number of Fibonacci additions: SC(n)=n–2 for $n \geq 2$, and 0 for n=0 or 1.

Q18. Why is the program's stepCount not 4n?

Because the code does not increment the counter four times per iteration. It executes stepCount++ only once per iteration, so its recorded count is n–2.

Q19. If four selected statements are counted per iteration, what is the count?

Under that explicitly chosen counting convention, the body contributes 4(n–2) statement executions. This is a different analytical count, not the value printed by this program's stepCount variable.

Q20. Does the loop contain exactly four operations?

It contains several statements and operations: addition, output, two assignments, and the counter increment, plus loop-control work. A precise count depends on what the examiner defines as one operation.

Q21. What is the difference between step count and time complexity?

Step count is a count under a specified operation-counting model. Time complexity describes how the amount of work grows as input size increases, usually using asymptotic notation.

Q22. What is the time complexity of this iterative program?

O(n), because the loop performs a number of iterations proportional to n and each iteration does a constant amount of work.

Q23. Why is the time complexity not O(4n)?

A constant factor does not change the asymptotic class: O(4n) simplifies to O(n). Big-O focuses on growth rate, not constant multipliers.

Q24. What is the auxiliary space complexity?

O(1), because the algorithm uses a fixed number of variables—n, a, b, c, i and stepCount—regardless of n.

Q25. What is the difference between O(1) space and O(n) space?

O(1) uses a fixed amount of extra memory. O(n) uses extra memory that grows proportionally with the input size.

Q26. What is an algorithm?

An algorithm is a finite, ordered set of clear steps for solving a problem or performing a computation.

Q27. What is DAA?

Design and Analysis of Algorithms is the study of designing efficient algorithms and evaluating their time and space requirements.

Q28. What is input size in this practical?

The input size is n, the number of Fibonacci terms requested.

Q29. What is best-case behaviour for this loop?

For $n \leq 2$, the loop performs zero iterations. For general complexity analysis over increasing n, the running time grows linearly.

C. Algorithm concepts and alternatives

Q30. What is a recurrence relation?

It defines a sequence or function using earlier values. Fibonacci uses $F(n)=F(n-1)+F(n-2)$.

Q31. What is iteration?

Iteration repeats a block of instructions using a loop, such as the for loop in this program.

Q32. What is recursion?

Recursion is a technique in which a function calls itself on smaller or simpler inputs until it reaches a base case.

Q33. How could Fibonacci be implemented recursively?

A function can return n when n is 0 or 1; otherwise it returns fib(n–1)+fib(n–2). This directly follows the recurrence relation.

Q34. Why is this iterative algorithm chosen?

It generates each term once, is easy to trace, avoids recursive function-call overhead, and uses constant auxiliary space.

Q35. What is the time complexity of naive recursive Fibonacci?

It is exponential, commonly bounded as O(2^n), because the same subproblems are recalculated many times.

Q36. What is the space complexity of naive recursive Fibonacci?

The maximum recursion depth is O(n), so the call stack can grow linearly with n.

Q37. What is memoization?

Memoization is top-down dynamic programming: store results of solved subproblems so they are not recalculated. Fibonacci can then be computed in O(n) time.

Q38. What is tabulation?

Tabulation is bottom-up dynamic programming: build answers from the base cases upward, storing earlier results in a table. It takes O(n) time and typically O(n) space.

Q39. Can dynamic programming reduce Fibonacci space usage?

Yes. Since each new term needs only the previous two, an iterative rolling-variable approach stores only two prior values and uses O(1) auxiliary space.

Q40. What is matrix exponentiation for Fibonacci?

It uses powers of a 2×2 transformation matrix to calculate the nth Fibonacci number in O(log n) time, typically using fast exponentiation.

Q41. Is matrix exponentiation better when printing the entire series?

Not necessarily. It can calculate a single distant Fibonacci number quickly, but printing all n terms requires at least O(n) output work.

Q42. What is the difference between an array-based and variable-based solution?

An array stores all generated terms and uses O(n) extra space. Three rolling variables store only the values needed next and use O(1) extra space.

Q43. Why might an array still be useful?

An array is useful when later terms must be accessed repeatedly, displayed by index, or reused in other calculations.

Q44. What is an advantage of the iterative approach?

It is simple, efficient, avoids repeated calculations, has O(n) time and O(1) auxiliary space.

D. Applications, limitations and improvements

Q45. What is a limitation of this program?

It does not validate negative input, uses a fixed-width int that can overflow, and its step counter counts only additions rather than every executed statement.

Q46. What is integer overflow in this program?

Overflow occurs when a Fibonacci value exceeds the range representable by int. The stored result can then become incorrect or cause signed-overflow undefined behaviour in C++.

Q47. How can overflow be reduced?

Use a wider integer type such as long long, while remembering that it also has a finite range. For very large values, use arbitrary-precision integers.

Q48. How should invalid input be handled?

Check that n is non-negative before generating the series. If n is negative, display an error message and stop or request valid input.

Q49. What does return 0 mean?

It indicates that main has completed successfully and returns a success status to the operating system.

Q50. What is Fibonacci search?

It is a searching technique for sorted arrays that uses Fibonacci numbers to choose probe positions. It is an alternative to binary search in certain settings.

Q51. What are Fibonacci heaps?

They are a family of priority-queue data structures that use Fibonacci-heap operations to achieve good amortized time bounds for operations such as decrease-key.

Q52. Where else are Fibonacci numbers used?

They appear in algorithm design and analysis, Fibonacci search and heaps, mathematical models, and patterns observed in some natural structures.

Q53. Are Fibonacci numbers a reliable financial prediction method?

No. Fibonacci ratios are used by some traders as technical-analysis reference levels, but they do not reliably predict market movements or guarantee outcomes.

Q54. Why is Fibonacci useful in algorithm analysis?

Its recurrence provides a simple example for studying recurrence relations, recursion trees, dynamic programming, and the difference between efficient and inefficient implementations.

Q55. What is the role of random access or indexing here?

The loop variable i tracks how many terms have been generated. The program prints terms sequentially and does not need random access to earlier terms.

Q56. How would you modify the program to calculate only the nth Fibonacci number?

Keep updating the two previous values until the required index is reached, then output the current value. This can still be done in O(n) time and O(1) auxiliary space.

Q57. How would you improve the practical program?

Validate n, use a suitable integer type or big integers, make the counter's definition explicit, and optionally separate Fibonacci generation into a function.

Q58. What is a base case in a recursive Fibonacci solution?

A base case is a stopping condition that returns a known value without another recursive call. For Fibonacci, fib(0)=0 and fib(1)=1 prevent infinite recursion.

Q59. Why can naive recursion repeat work?

For example, fib(5) calls fib(4) and fib(3), while fib(4) also calls fib(3). The same values are calculated multiple times; memoization or tabulation avoids this repetition.

Q60. What should you say if asked whether the step count is n−2 or 4(n−2)?

For this code's printed stepCount, it is n−2 because one increment occurs per loop. If an examiner defines four specific statements as counted operations, their separate statement count is 4(n−2). Always state the counting convention.

Last-minute recall

Topic	Remember
Fibonacci rule	Add the previous two terms.
Initial terms	0 and 1.
Loop	Starts at i=2; runs while i<n.
Loop executions	n−2 when n≥2.
Code's stepCount	One increment per iteration = n−2.
Four-statement count	4(n−2), only if that counting convention is specified.
Time complexity	O(n).
Auxiliary space	O(1).
Naive recursion	Exponential time; O(n) call stack.
Memoization / tabulation	O(n) time; avoids repeated subproblems.
Matrix exponentiation	O(log n) for one nth term.